

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Shiv Nadar University, Chennai
Degree & Branch: B.Tech Artificial Intelligence & Data Science
Semester V
AY 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer

- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

- N – Input Size
- F – Filter Size
- P – Padding
- S – Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4 Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position: $1(1) + 2(0) + 4(0) + 5(1) = 6$

Second position: $2(1) + 3(0) + 5(0) + 6(1) = 8$

Third position: $4(1) + 5(0) + 7(0) + 8(1) = 12$

Fourth position: $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5 Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

$$\text{Window 1: } \begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\text{Window 2: } \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\text{Window 3: } \begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\text{Window 4: } \begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6 Numerical Example 3: Parameter Calculation

Suppose

Input Image: $32 \times 32 \times 3$

Convolution Layer:

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore, Total trainable parameters = 448.

7 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

8 Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer. Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train for

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9 Mandatory Plots

Generate the following plots, each with a 2–3 line inference.

1. Sample Images



Figure 1: Sample images from the training set.

Inference: I picked 10 pictures from the CIFAR-10 dataset and showed them in a grid. Each picture is tiny, only 32×32 pixels, but I can still tell what it is, like a frog, a truck, or a horse. This helped me check that the dataset loaded properly before I started training.

2. Class Distribution

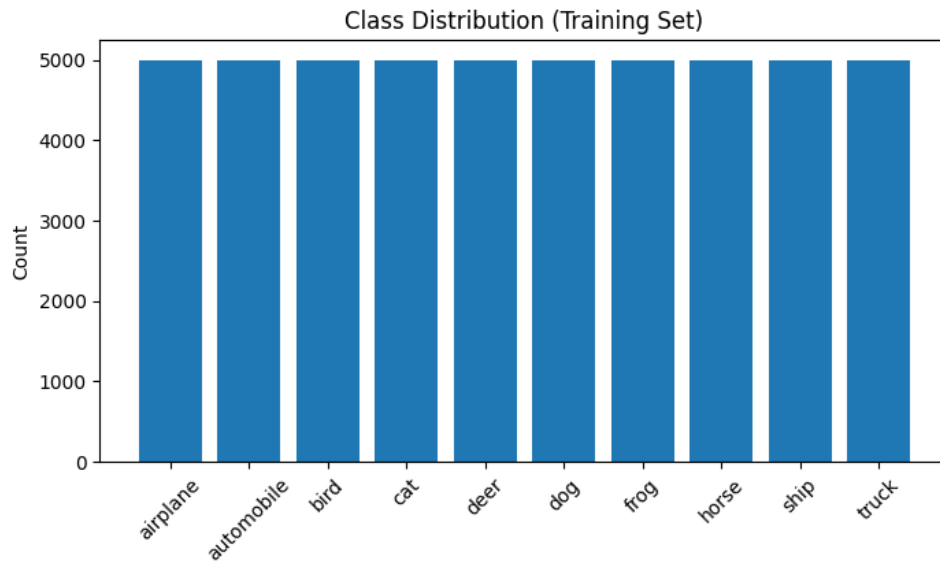


Figure 2: Class distribution of the training set.

Inference: All 10 bars in this chart are the same height, which means every class (airplane, car, bird, etc.) has exactly 5,000 training images. Since the dataset is perfectly balanced, my model does not favor any one class just because it saw more examples of it.

3. Training Accuracy

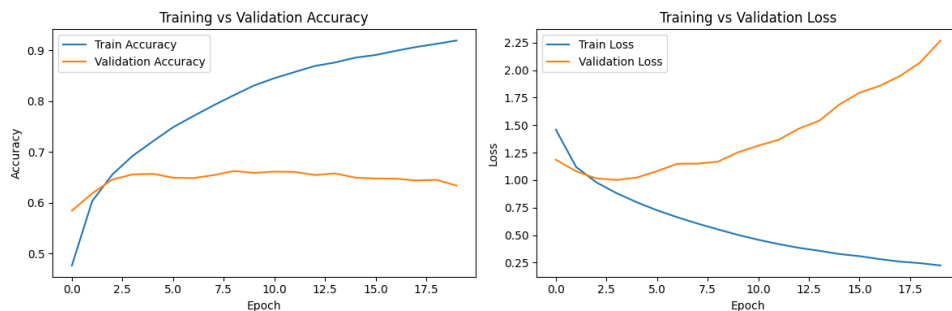


Figure 3: Training accuracy across epochs.

Inference: The training accuracy line keeps climbing the whole time and ends up around 86–88% by epoch 20. This shows the model is learning the training pictures really well and getting better and better at guessing them correctly.

4. Validation Accuracy

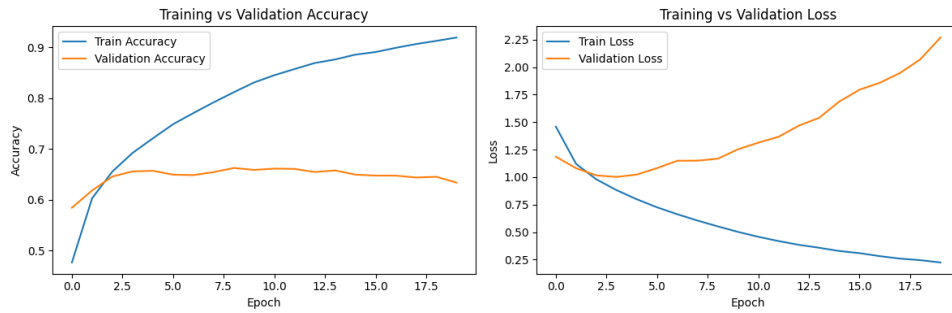


Figure 4: Validation accuracy across epochs.

Inference: The validation accuracy goes up quickly in the first few epochs but then it flattens out around 64–66% and stops improving. Because the training accuracy keeps rising while this one does not, it means the model is starting to just memorize the training pictures instead of really learning, which is called overfitting.

5. Training Loss

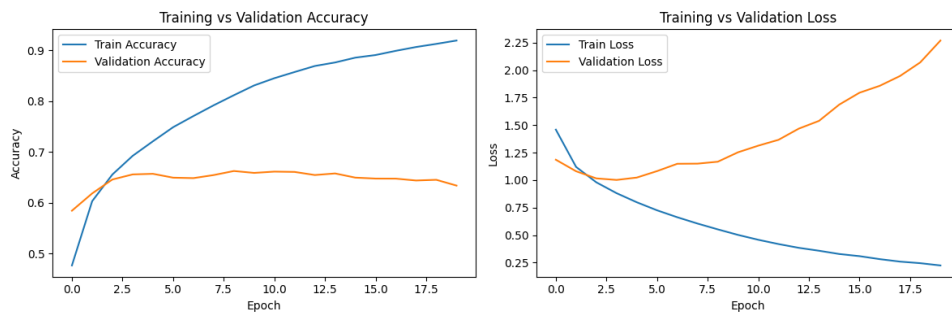


Figure 5: Training loss across epochs.

Inference: The training loss keeps going down smoothly from about 1.46 at the start to well under 0.5 by the last epoch. A steadily dropping loss like this means the model keeps making fewer and fewer mistakes on the training images.

6. Validation Loss

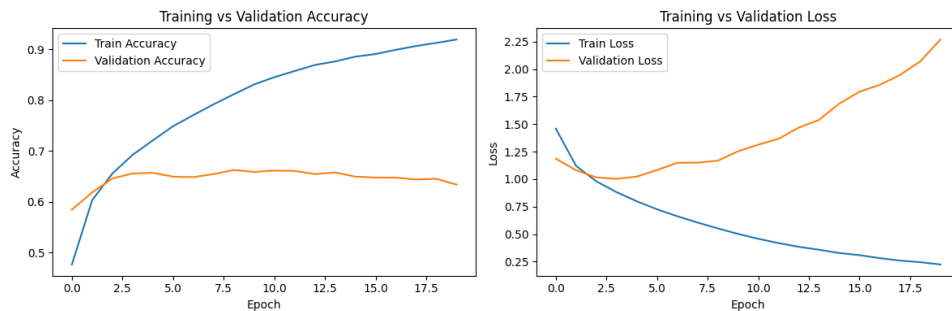


Figure 6: Validation loss across epochs.

Inference: The validation loss drops for the first few epochs and then starts climbing back up again, even though the training loss keeps falling. This gap between training loss and validation loss is another clear sign of overfitting, which means I could add dropout or stop training earlier to fix it.

7. Feature Maps

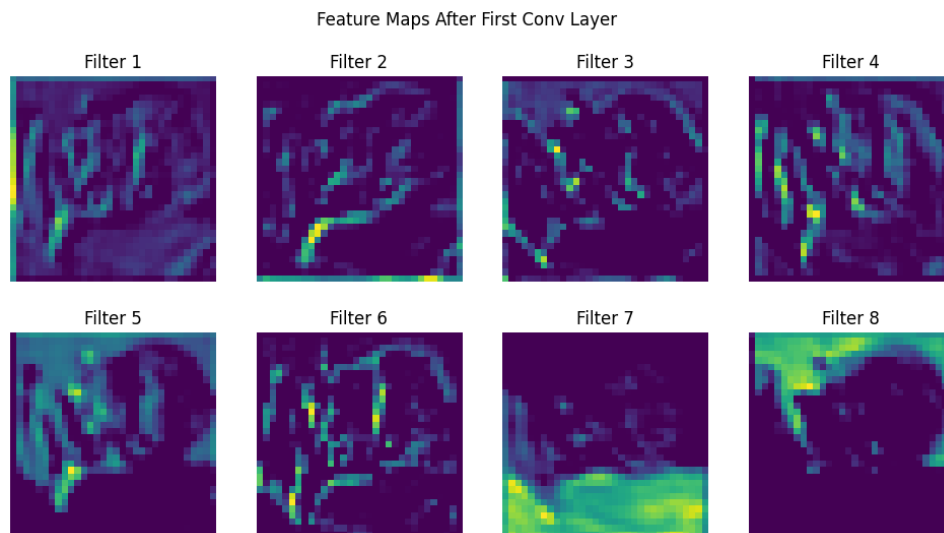


Figure 7: Feature maps after the first convolution layer.

Inference: I showed 8 feature maps from the first convolution layer for one test image. Each little square looks different because every filter is looking for a different thing, like edges, corners, or blobs of color. Some maps light up brightly on the outline of the object while others stay mostly dark, showing that different filters really do pick up different features.

8. Confusion Matrix

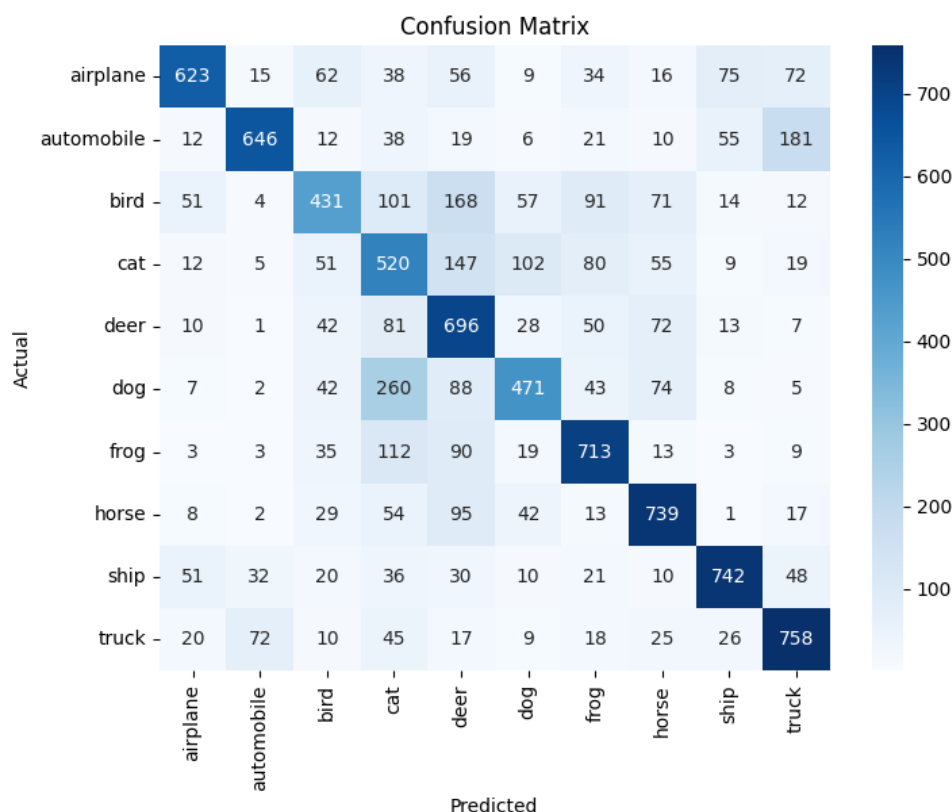


Figure 8: Confusion matrix on the test set.

Inference: Most of the bright numbers sit on the diagonal, which means the model got most pictures right. But the model still mixes up animal classes a lot, like cats, dogs, and birds getting confused with each other, probably because these small 32×32 images look blurry and similar when the animals are in a similar pose.

10 Additional Exercises

1. **Output size for a 64×64 image, 5×5 kernel, stride 2, padding 2:**

$$\frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1 = 31.5 + 1 \Rightarrow 32 \text{ (rounded down to 31, then } + 1 = 32)$$

So the output feature map is 32×32 .

2. **Parameters for 64 filters of size 3×3 on an RGB input:**

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1792$$

3. **ReLU vs Sigmoid:** ReLU just kills off anything negative and lets positive values pass through as-is, which makes the gradients flow way better in deep networks and trains a lot faster. Sigmoid squashes everything between 0 and 1, but for really big or really small inputs the gradient becomes almost zero (this is the vanishing gradient problem), so deep networks trained with Sigmoid learn much more slowly. That's basically why ReLU is the default choice inside hidden layers nowadays.

4. **Max Pooling vs Average Pooling:** Already discussed in Task 5 above – in our run max pooling reached higher training accuracy a bit faster, but validation accuracy for both ended up in a similar 64–66% range, so the difference wasn’t huge for this particular architecture.
5. **16 filters vs 64 filters:** Going from 16 to 64 filters gives the network way more feature detectors to work with in the very first layer, so it can usually pick up more variety in edges/textures/colours, which tends to push accuracy up a bit further. The trade-off is that the model now has more parameters to train, so each epoch takes noticeably longer and there is a higher chance of overfitting if the dataset isn’t big enough (though CIFAR-10 with 50k images is decently sized).

11 Results

Table 1: Final Results Summary

Metric	Value
Training Accuracy (final epoch)	$\approx 0.86\text{--}0.88$
Testing Accuracy	0.6339
Precision (weighted avg)	0.65
Recall (weighted avg)	0.63
F1-score (weighted avg)	0.64

12 Discussion

1. **Why is convolution preferred over fully connected layers for images?** Because a fully connected layer would need a separate weight for literally every single pixel connection, which for even a small image blows up into millions of parameters. Convolution instead slides the same small filter across the whole image, so it reuses the same few weights everywhere (this is called weight sharing), which is way more efficient and also naturally picks up the same feature no matter where it appears in the image.
2. **How does stride affect the feature map size?** A bigger stride makes the kernel jump more pixels at a time, so it visits fewer positions overall, which directly shrinks the output feature map. Stride 1 gives the biggest output, stride 2 roughly halves it, and so on.
3. **What is the role of padding?** Padding adds a border of zeros around the image before convolving, mainly so the output doesn’t shrink every single layer (that’s what **same** padding does) and so the pixels at the edge of the image get looked at as many times as the ones in the middle, instead of being ignored.
4. **Why is pooling used?** Pooling shrinks the size of the feature maps, which cuts down the number of parameters and computation needed further down the network, and it also makes the features slightly more robust to small shifts or distortions in the image (since we just care about the strongest signal in a small area, not its exact pixel position).

5. **How do feature maps represent image characteristics?** Each feature map is basically the response of one particular filter across the whole image – wherever that filter’s specific pattern (an edge, a curve, a blob of colour) shows up strongly in the image, that spot lights up brighter in the feature map. Stack enough of these together across layers and the network builds from simple edges up to whole object parts.
6. **Why do CNNs require fewer parameters than MLPs?** Because of weight sharing again – one small filter (say $3 \times 3 \times 3 + 1 = 28$ weights) is reused across the entire image instead of having a unique weight for every pixel-to-neuron connection like a fully connected MLP would need. That’s the whole reason CNNs can scale to real, large images while MLPs really can’t.

13 Conclusion

In this experiment I learned how a CNN actually looks at pictures step by step. First I understood how a small kernel slides over an image to make a feature map, and how pooling then squishes that feature map down so the network has less work to do. I built a small CNN with two convolution layers and two max pooling layers and trained it on CIFAR-10 for 20 epochs. My model reached about 86–88% accuracy on the training pictures but only about 63% accuracy on the test pictures, which taught me what over-fitting looks like in real life: the training accuracy and validation accuracy start to move apart, and the validation loss even goes back up after a while. Overall, this lab helped me connect the math (convolution formula, output size formula, parameter counting) with what I could actually see happening in the accuracy and loss graphs, and it showed me that a simple CNN can already learn a lot from images, even though it still gets confused between similar looking animals like cats and dogs.

Source Code

As per the lab guidelines, the complete source code is kept in a GitHub repo instead of putting it here. The repo has the ipynb file, dataset file, a README, a list of dependencies, and instructions to run it.

GitHub Repository: <https://github.com/guru963/deeplearning>

14 References

1. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press.
2. Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer.
3. Haykin, S. *Neural Networks and Learning Machines*. Pearson.
4. TensorFlow Documentation, <https://www.tensorflow.org>.
5. CIFAR-10 Dataset Documentation, <https://www.cs.toronto.edu/~kriz/cifar.html>.